Deep Research Request: Preflight — Vibe Coder Project OS

Context

I am building an open-source web application called **Preflight** — a project operating system designed specifically for "vibe coders" (people who build software using AI coding assistants like Lovable, Bolt, Cursor, Claude Code, Replit, v0, and similar platforms). The goal is to eliminate chaotic workflows and replace them with a structured, AI-ready build pipeline that takes a user from raw idea → production-ready app.

Preflight generates: deep research prompts, UI design prompts, coding agent system instructions, rules files, and a full sequential build prompt workflow — all scoped to the user's specific project.

Research Objectives

1. Market & Audience Analysis

- Who exactly are "vibe coders"? Define the demographic, technical skill level, and behavioral profile. What platforms do they predominantly use?
- What is the current and projected market size of the no-code / AI-assisted development space (2024–2027)?
- What are the top 5–10 most painful friction points vibe coders experience when building apps with AI tools? (Context loss, broken prompts, lack of structure, design inconsistency, documentation gaps, etc.)
- How do vibe coders currently manage their projects, notes, prompts, and assets? What tools do they use (Notion, Obsidian, plain text files, etc.)?
- What communities exist around vibe coding (Reddit, Discord, X/Twitter, YouTube)? What topics generate the most engagement?

2. Competitive Landscape

- Are there any existing tools that specifically serve as "pre-build" or "project OS" layers for AI-assisted developers? (Tools that organize ideas, generate prompts, structure workflows before the actual coding begins.)
- Analyze the following tools and identify their gaps relative to Preflight's use case:

    - Notion / Obsidian (general note/project management)
    - Linear / Jira (dev project management)
    - PromptBase / FlowGPT (prompt libraries)
    - Magick / n8n / Flowise (visual AI workflow builders)
    - Any emerging "AI project starter" or "prompt orchestration" tools
- What would make a developer-focused project OS tool go viral in the vibe coding community?

3. Technology Stack Recommendations

For a web-first application that must be:

- Portable (can be tested on Lovable, Bolt, Cursor, Claude Code, Replit, v0)
- Local-first (IndexedDB / localStorage for open-source, no-auth version)
- Cloud-optional (Supabase as the default hosted backend)
- Open-source friendly (MIT or Apache 2.0 license, easy to self-host)

Research and recommend:

- Frontend framework best suited for this (React + Vite vs Next.js vs SvelteKit) — focus on what generates the cleanest output from AI coding tools
- State management approach for local-first data (Zustand vs Jotai vs Redux vs TanStack Query)
- Local persistence layer (Dexie.js + IndexedDB vs localStorage vs PGlite)
- Optional cloud sync architecture (Supabase Realtime vs CRDTs like Yjs vs ElectricSQL)
- Best libraries for copy-to-clipboard, markdown rendering, file upload/download in a React app
- How to architect a BYOK (Bring Your Own Key) AI integration that supports multiple providers (Claude, OpenAI, Gemini) with a clean abstraction layer

4. Design System Research

Research design patterns and UI systems specifically for:

- **Developer tools / productivity apps** — what design patterns reduce cognitive load? (Linear, Raycast, Arc Browser, Vercel dashboard, Supabase dashboard)
- **Dark-mode-first interfaces** — what color systems, typography choices, and spacing scales work best?
- **Glassmorphism + premium dark aesthetic** — how to implement this in 2024/2025 Tailwind CSS without performance issues?
- **Command palette pattern** (Cmd+K) — is this essential for a developer tool? Best React libraries for implementing it (cmdk, kbar)?

- **Onboarding UX for productivity tools** — what is the most effective pattern for a "first launch" experience that converts users into active daily users?
- **Mobile-responsive developer tools** — is it expected? What level of mobile support is standard for this category?
- What are the best Tailwind UI component libraries for building a premium dark developer tool? (shadcn/ui, Radix UI, Tremor, etc.)

5. Prompt Engineering Architecture

- What are the current best practices for writing sequential, chained AI coding prompts that don't break existing functionality? (Research specifically for Lovable, Cursor, Claude Code, and Bolt)
- How do top "super-prompters" structure their system instructions for coding agents to ensure: modular code, no oversized files, mandatory documentation updates, test-first approach, and scalability?
- What is the optimal structure for a `.cursorrules` / `CLAUDE.md` / `system_instructions.md` file for a full-stack React + Supabase project?
- Research the concept of "prompt chaining" vs "prompt batching" — which approach produces better results with current coding agents?
- Are there any open-source prompt libraries or frameworks specifically for structured software development workflows?

6. Open Source Strategy

- What makes an open-source developer tool project go from 0 to 1,000 GitHub stars quickly? What are the key factors? (README quality, demo video, Hacker News launch, Product Hunt, etc.)
- What license is most appropriate for a project that is open-source on GitHub but also has a paid hosted version? (MIT vs Apache 2.0 vs BSL vs AGPL)
- What monetization models work best for "open-core" developer tools? (Self-hosted free + hosted paid, BYOK free + managed API paid, etc.)
- What are the top 5 recently successful open-source developer tool launches to study?

Output Format Requirements

Please structure your research report as follows:

1. Executive Summary (key findings, 1 page)
2. Market & Audience (detailed findings + data)
3. Competitive Landscape (comparison table + gap analysis)
4. Technology Recommendations (specific stack with justification)
5. Design System Guidance (with examples and references)

6. Prompt Engineering Best Practices
7. Open Source Launch Strategy
8. Key Risks & Blind Spots
9. Recommended Next Steps

Please include sources, data points, and specific tool/library names wherever possible. Prioritize findings from 2024–2025.

1. Executive Summary (key findings, 1 page)
   Preflight sits in a fast-growing, under-instrumented niche: AI-assisted "vibe coding" where semi-technical builders and engineers drive app creation primarily through natural language in tools like Lovable, Bolt, Cursor, Replit, and v0. These platforms provide strong code generation but weak project-level structure, leaving users to glue together prompts, notes, and decisions across Notion, ad-hoc files, and chat history.[1] [2] [3]

The broader AI software and generative-AI markets are growing at roughly 25–33% annually, with generative AI expected to reach around 200–220B USD by 2030, implying a rapidly expanding addressable audience for AI-first dev tooling between 2024–2027. Within dev tools, AI-native editors and open-source platforms like Cursor, Supabase, and Ollama are among the fastest-growing projects, and GitHub stars plus community traction function as core validation signals.[4] [5] [6] [7]

Vibe coders' main pain points cluster around: context fragmentation between tools; brittle prompts that don't encode project rules; difficulty enforcing modular, small-PR workflows with AI agents; inconsistent UI/UX and lack of design guidance; and missing, up-to-date documentation. They currently manage projects with generic knowledge tools (Notion, Obsidian), lightweight task systems, and the built-in project views of their AI IDEs, plus Reddit/Discord/X for pattern discovery and troubleshooting.[2] [6] [8] [9] [10] [11]

There is no dominant, dedicated "pre-build OS" for AI-assisted projects; instead, builders use a patchwork of note apps, prompt libraries (PromptBase, FlowGPT), and visual orchestration tools (n8n, Magick, Flowise) that focus more on runtime flows than on pre-coding project structure. This leaves a gap for Preflight as a structured, opinionated "project brain" that generates and maintains: research prompts, UI prompts, agent instructions, rules/config files, and sequential build prompts tailored to each app.[12] [13]

Technically, a React + Vite or Next.js + App Router + TypeScript + Tailwind + shadcn/ui stack maps best to AI-generated code quality and developer expectations, with Dexie.js or RxDB on top of IndexedDB for local-first data, optional ElectricSQL/Supabase for sync, and a thin BYOK abstraction for OpenAI/Claude/Gemini. On the design side, the winning pattern is a premium dark, keyboard-driven productivity UI inspired by Linear/Raycast/Vercel, with command palette (cmdk or kbar), dense but clean layouts, and a small, consistent spacing and color scale (e.g., Radix Colors + Tailwind).[14] [15] [16] [17] [13] [18] [19] [12]

Prompt-engineering best practice for tools like Cursor, Lovable, and Claude Code emphasizes: short, scoped tasks; explicit phases (analyze → plan → implement → test → docs); lean but stable rules files (.cursorrules, CLAUDE.md); and clear separation between persistent project rules and ephemeral context. Open-source success for Preflight will hinge on: an exceptional

README and demo video, focused positioning around "AI project OS for vibe coders," a well-timed launch on Hacker News/Product Hunt, and an open-core model (MIT/Apache for the app, paid hosted sync/collab).[6] [20] [21] [22] [23] [24] [25] [9] [26] [10] [11]

2. Market & Audience (detailed findings + data)

Who are "vibe coders"?

- Demographic and skill profile:
  - Mix of indie hackers, startup founders, product-minded engineers, and advanced no-coders who are comfortable with web concepts but prefer natural-language workflows over raw coding.[3] [8] [2]
  - Many come from JavaScript/React/Next backgrounds or from nocode tools (Webflow, Bubble, Retool) stepping into AI-assisted code editors.[2] [3]
  - High time pressure, strong bias to ship MVPs quickly, and willingness to accept some tech debt in exchange for speed.[3] [2]
- Behavioral traits:
  - Build in "sessions": intensive multi-hour bursts in Cursor/Lovable/Bolt, driven by chat prompts and "do this next" instructions.[9] [10] [11] [2]
  - Frequently context-switch between editor, browser, docs, and chat, leading to lost decisions and duplicated instructions.[10] [9]
  - Heavy use of templates, starter kits, and copied prompts; most do not build their own formal project rule sets unless forced.[11] [9] [10]
- Primary platforms used:
  - Editor / environment: Cursor, Lovable, Replit, Bolt.new, v0, and sometimes VS Code + AI extensions (Copilot, Codeium, Cody).[6] [2] [3]
  - Backend & infra: Supabase/Mongo/PlanetScale for databases; Vercel/Netlify/Replit hosting; n8n/make.com/Flowise for integrations & automations.[13] [19] [12] [2] [6]

Market size (no-code / AI-assisted development, 2024–2027)

- AI software overall:
  - ABI Research estimates AI software at about 122B USD in 2024, projected to 174.1B in 2025, growing to 467B in 2030 at ~25% CAGR.[4]
  - Another report pegs the broader AI market at 224.41B in 2024, reaching 1.24T by 2030 at 32.9% CAGR.[5]
- Generative AI / dev tooling slice:
  - ABI estimates generative AI growing from 37.1B in 2024 to 220B by 2030 (~29% CAGR), with foundation models and dev-tooling being key segments.[4]
  - Open-source dev-tools analysis shows ~48% of early-stage dev-tool funding in 2024 going to AI-enabled tools, with projects like Cursor and Ollama among the fastest-growing by stars and funding.[6]

Interpretation:

- The specific "AI-assisted development" niche is not always isolated in market reports, but the funding and usage signals show that AI-native dev environments and local-first tooling are one of the hottest sub-segments.[6]
- Reasonable assumption: the reachable market for Preflight (AI dev tools users plus advanced no-coders) grows at least in line with generative AI (~29% CAGR) between 2024–2027, with early adopters concentrated in North America and quickly growing in APAC/Europe.[7] [5] [4]

Top friction points for vibe coders

From platform docs, community posts, and "prompting handbook" style guides, several recurring pain points stand out:

- Context fragmentation:
  - Important decisions live in scattered places: chat threads inside Cursor/Lovable, ad-hoc Markdown files, Notion docs, and Discord messages.[9] [10] [11] [2]
  - AI agents often "forget" rules or project background when context windows reset or files are not explicitly referenced.[20] [23] [25] [10]
- Broken or brittle prompts:
  - Best-practice guides stress repeating critical instructions and avoiding "do 5 things at once," implying that single, overloaded prompts regularly fail.[26] [10] [11] [9]
  - Users struggle with prompt reuse and versioning; patterns are copied between projects without adaptation, causing inconsistent behavior.[26] [11] [9]
- Lack of structure and workflow:
  - Tools like Lovable and Cursor offer modes (plan, visual edits) but do not enforce a project-wide workflow: analyze → design → implement → test → document.[10] [2] [9]
  - Super-prompters often manually enforce modes (e.g., separate analysis and implementation prompts, multi-phase tasks), which most users do not formalize.[9] [26] [10]
- Design inconsistency:
  - Many vibe-coded apps mix components from different UI kits and AI-generated CSS, leading to visual debt and friction when editing later.[16]
  - There is little guidance inside AI IDEs about consistent spacing, color systems, or component usage, pushing users to external design systems.[15] [14] [16]
- Documentation gaps and drift:
  - AI can generate docs and READMEs, but nobody keeps them in sync; few tools automatically enforce "update docs/tests as part of each change."[23] [25] [20] [26] [10]
  - Community advice often recommends adding documentation tasks explicitly to prompts, indicating missing first-class support.[11] [26] [10]
- Code quality & scalability:

- AI code often arrives as large, monolithic files; super-prompter posts emphasize instructions for small files, modular architecture, and tests, which default tools do not encode.[26] [9]

Current project management practices & tools

- Workspace management:
  - Many AI builders rely on generic tools like Notion and Obsidian to structure ideas, backlogs, and prompt snippets; these are not tightly integrated with their AI IDEs.[16] [6]
  - Some use VS Code workspaces and repo-level docs, but vibe coders in Lovable/Cursor often keep everything in the project repository or platform UI.[27] [2] [3]
- Prompt & asset management:
  - Prompt libraries like PromptBase and FlowGPT are used to discover and copy prompts, but they operate as external marketplaces, not project-specific OSes.[12] [13]
  - Many users keep their own "prompt docs" or wiki pages in Notion/Markdown to reuse patterns across builds.[10] [9] [26]

Communities and active topics

- Community hubs:
  - Reddit: r/Cursor, r/ClaudeCode, r/SaaS, r/LocalLLaMA, and vibe-coding specific threads discuss prompts, rules files, and workflow hacks.[8] [24] [25] [9] [26]
  - Discord: Official tool communities (Cursor, Supabase, Lovable, Replit) plus AI-builder servers; topics include prompt recipes, system config, and local-first setups.[13] [12] [6]
  - X/Twitter and YouTube: Ranking videos of AI coding tools and devlog-style content on building full apps with Cursor/Lovable/Replit "from a prompt."[1] [27] [3]
- Topics that drive engagement:
  - "My .cursorrules / CLAUDE.md setup," "23 prompts I use for flawless Cursor code," "I shipped an MVP in a weekend with Lovable."[25] [11] [9] [26] [10]
  - Open-source success stories: "0 to 1k GitHub stars in a week," and deep dives into monetizing dev-tools.[21] [22] [24] [6]

3. Competitive Landscape (comparison table + gap analysis)

High-level competitor roles

Tool / Category	Core purpose	Strengths	Limitations vs Preflight's vision
Notion / Obsidian	Generic knowledge & project management	Flexible docs, databases, personal knowledge management	Not AI-dev specific; no first-class concept of prompts, rules files, build phases. [16]
Linear / Jira	Issue & sprint management	Great for teams, prioritization, analytics	Optimized for human-written tickets, not AI-prompt pipelines or rules generation.

Tool / Category	Core purpose	Strengths	Limitations vs Preflight's vision
PromptBase / FlowGPT	Prompt discovery & marketplace	Large community of prompts and templates	Not project-scoped; little support for versioned, app-specific prompt workflows. [12]
Magick / n8n / Flowise	Visual AI & automation flows	Low-code orchestration, connectors, automations	Focus on runtime workflows; not focused on pre-build planning or IDE-integrated prompts. [13] [11]
Cursor / Lovable / Bolt	AI-native code editors & app builders	Strong coding assistance, templates, integrated hosting	Little structured "project OS" before coding; rules files exist but must be hand-crafted. [2] [9] [10] [11]

Existing "pre-build / project OS" offerings

- There are growing tutorials and tools that help optimize config files like .cursorrules, CLAUDE.md, and copilot instructions, but these focus on individual files, not an integrated OS across research, design, and build.[28] [20] [23] [25] [9]
- Local-first databases like Dexie and RxDB and sync services like ElectricSQL provide the underlying tech for a local-first project OS but are not themselves structured workflow tools for AI builders.[19] [12] [13]

Gap: No widely adopted app makes "project-scoped prompt / rules / workflow design" its primary product. This is the slot Preflight can own.

What would make a project OS go viral among vibe coders?

- Clear promise & positioning:
  - "Never lose your project's brain again" or "One OS for your prompts, rules, and build steps, wired for Cursor/Lovable/Supabase."
- Tight alignment with existing workflows:
  - Export one-click: generate .cursorrules, CLAUDE.md, Supabase schema prompts, UI prompts, and task prompts tuned for a specific stack.[20] [28] [23] [25] [10]
  - Paste-friendly outputs designed for Lovable Plan mode, Cursor tasks, Bolt flows, etc. [11] [9] [26] [10]
- Strong defaults & templates:
  - Opinionated templates: "Full-stack React + Supabase SaaS," "Multi-tenant dashboard," "AI-powered CRUD app," each with pre-built rules and build sequences.[19] [16] [12]
  - Example .cursorrules and CLAUDE.md tuned for vibe coding best practices.[23] [25] [20] [9] [26]
- Launch mechanics:
  - Highly polished README and animated demo showing a messy idea turned into a structured build run end-to-end.[24] [21] [6]
  - Posting in relevant communities (r/Cursor, r/ClaudeCode, HN, Product Hunt) with real stories of shipping faster.[29] [24]

4. Technology Recommendations (specific stack with justification)

Frontend framework

- React + Vite vs Next.js vs SvelteKit:
  - AI-IDE ecosystem: Most AI coding environments and templates assume React/TypeScript, and many community examples use Tailwind + shadcn + Radix.[27] [2] [16] [6]
  - Next.js (App Router): Excellent for long-term scalability, routing, and SSR, but Preflight's initial offline/local-first focus and "no-auth" mode favors a purely client-side bundle.
  - For an open-source, local-first tool expected to be run inside AI editors, React + Vite + TypeScript is a strong default: simple, fast dev build, minimal magic, ideal for AI to modify.[16]

Recommendation:

- Phase 1 (local-first, open-source): **React** + **Vite** + **TypeScript** + **Tailwind** to maximize compatibility with AI editor templates and keep mental overhead low.[16]
- Phase 2 (hosted, cloud-optional version): Optionally migrate to Next.js or keep Vite + a small Express/Edge backend alongside Supabase.

State management for local-first data

- Options: Zustand, Jotai, Redux, TanStack Query.
  - TanStack Query excels at server state but is less necessary for purely local IndexedDB-backed state.
  - Redux is heavyweight for a greenfield tool.
  - Zustand and Jotai integrate well with React and are used in modern AI-tool examples; Zustand is particularly ergonomic for local-first stores.[13] [16]

Recommendation:

- Use **Zustand** for app-state (projects, prompts, UI state) with middleware for persistence to Dexie/IndexedDB.[12] [13]
- Introduce TanStack Query later only if/when remote APIs (Supabase/ElectricSQL) are involved heavily.

Local persistence & sync

- Local storage layer:
  - localStorage is trivial but insufficient for structured, syncable, multi-MB data.
  - IndexedDB wrappers like Dexie abstract away complexity and support offline-first workflows, and Dexie now advertises unified data layers with optional sync.[12]

    - RxDB offers a richer local-first DB with sync, encryption, and reactive queries, suitable if you envision collaborative features later.[13]

Recommendation:

- **Dexie.js** + **IndexedDB** as the first local database: simple, well-documented, AI-friendly, and enough for offline projects.[12]
- Keep the data model close to relational/JSON documents to ease future sync to Supabase/ElectricSQL.[19] [12]
- Optional cloud sync:
    - Supabase provides Postgres, auth, file storage, and Realtime for live updates.[19] [6]
    - ElectricSQL can layer local-first sync semantics on top of existing Postgres APIs, enabling low-latency collaboration.[19]
    - CRDT-based solutions (Yjs) are ideal for collaborative text editing, but Preflight's core data is structured prompts/rules which can live happily in Postgres + Electric.

Recommendation:

- "Cloud optional" architecture:
    - Core app runs entirely on Dexie/IndexedDB.[12]
    - When user enables sync, project entities replicate to **Supabase** tables, and an optional **ElectricSQL** layer handles local-first sync for multi-device or collaborative usage.[13] [19]

Utilities: clipboard, markdown, files

- Copy-to-clipboard:
    - `navigator.clipboard.writeText` wrapped in a simple hook, or a small library like `use-clipboard-copy` if you want ergonomics. (Common practice in React apps.)
- Markdown rendering:
    - `react-markdown` plus `remark-gfm` is a widely adopted stack to render GitHub-style markdown.
    - For syntax highlighting, pair with `react-syntax-highlighter` or `shiki`-based rendering.
- File import/export:
    - Use the File System Access API where supported, and fall back to hidden file inputs and Blob downloads (`URL.createObjectURL`).
    - For a nicer UX, wrap these in custom hooks to export/import full Preflight project JSON and generated config files.

BYOK AI integration (OpenAI, Claude, Gemini, etc.)

Architectural goals: clear abstraction, easy to add providers, and straightforward for AI coding tools to modify.

- Core design:

    - Define a **Provider** interface, e.g.:
        - `complete({ model, system, messages, temperature, ... })`
        - `streamComplete(...)`
    - Implement adapters per vendor: `OpenAIProvider`, `AnthropicProvider`, `GoogleProvider`, each mapping this interface to their HTTP APIs.
    - Store keys locally (localStorage/IndexedDB) in the browser-only build; never send keys to your backend unless user chooses cloud mode.
- Key patterns:
    - Use vendor-agnostic model IDs at your layer ("gpt-4.1-mini", "claude-3.7-sonnet", "gemini-1.5-pro") and have each adapter translate them to API-specific endpoints.
    - Keep all prompt templates in versioned JSON/TS objects so Preflight itself can be used via AI tools to modify the prompt architecture.

5. Design System Guidance (with examples and references)

Developer-tool UX patterns

- Inspiration set: Linear, Raycast, Vercel, Supabase dashboards.[14] [15] [16]
- Common traits that reduce cognitive load:
    - High information density but with clear hierarchy (sidebar navigation, top command bar, central work area).[16]
    - Keyboard-first navigation (Cmd+K palette, global shortcuts for switching projects/modes).[17] [18] [30]
    - Reduced chrome: minimal borders, subtle shadows, and tight spacing (4/8/12/16 px).[15] [16]

Example: A Preflight workspace with left sidebar (Projects, Spaces, Rules, Prompts), main panel showing the current project pipeline, and a right drawer for context and generated outputs mimics Linear's cognitive model.

Dark-mode-first choices

- Color systems:
    - Radix Colors provide well-tuned neutral and accent scales with both light and dark tokens, making it easy to keep contrast consistent.[15] [16]
    - Libraries like shadcn/ui ship Tailwind configs aimed at dark-mode support out of the box.[14] [16]
- Implementation:
    - Use CSS custom properties for colors and let Tailwind reference them, as suggested in dark-mode guides for Radix + Tailwind.[15]
    - Define a small palette: `bg`, `bg-elevated`, `border`, `text`, `subtle`, and 1–2 accent colors; avoid "rainbow" UIs.[15] [16]

Glassmorphism + premium dark aesthetic

- To avoid performance issues:
  - Use `backdrop-blur` and semi-transparent layers sparingly (e.g., command palette, modals), not on large, scrollable surfaces.[17]
  - Prefer fewer layers and rely on subtle shadows and overlays rather than multiple nested blurred containers.[15]
  - Tailwind supports `backdrop-blur-sm/md` plus opacity utilities, which are hardware-accelerated in modern browsers.[16] [15]

Suggested pattern:

- Solid `bg-slate-950` base, slightly lighter `bg-slate-900/80` for panels, blur only on overlays (`backdrop-blur-md bg-slate-900/80`).[15] [16]

Command palette (Cmd+K)

- Importance:
  - Command palettes are becoming table stakes for modern dev-tools (Raycast-like interfaces); they greatly reduce navigation friction and feel natural to vibe coders already using AI editors.[18] [30] [17]
- Libraries:
  - `cmdk` and `react-cmdk` provide accessible components for building command palettes, optimized for React.[30] [18] [17]
  - `kbar` offers a higher-level, unstyled command palette with built-in hotkeys, fuzzy search, and virtualization.[17]

Recommendation:

- Use **cmdk or react-cmdk** for a Linear-style palette, integrated with your routing and key workflows (e.g., "Generate next build prompt," "Open Rules," "Export .cursorrules").[18] [30] [17]

Onboarding UX for productivity tools

- Effective patterns observed across dev tools:
  - "Guided first project" that creates a sample workspace and walks users through 3–5 steps (define idea → choose stack → generate first prompts).
  - Short, in-UI tutorial checklists rather than modal walls of text.
  - Embedded "Try it now" examples where users can inspect a sample Preflight project and its generated rules.
- Borrow from Product Hunt / OSS success posts: showing a "wow" end state quickly is critical for adoption.[24] [6]

Mobile support expectations

- Most dev-tools are desktop-first; mobile is often limited to read-only dashboards or quick edits.[16]
- For Preflight, mobile should be considered "nice to have, read-friendly" (view project plan, notes) rather than "full editing," which is keyboard-heavy anyway.

Tailwind UI component libraries

- shadcn/ui: Tailwind + Radix-based component library popular in React/Next projects, with strong dark-mode support and dev-tool aesthetics.[14] [16]
- Radix UI: Low-level primitives for accessible popovers, dialogs, etc., great for building your own premium dark components.[15] [16]
- Others: Tremor focuses on dashboards and data-viz UIs, useful if you add analytics about prompt usage or agent performance.

Recommendation:

- Base on **shadcn/ui** + **Radix** plus your own tokens for colors and spacing; this will also align well with the UI components AI tools commonly generate in 2024–2025.[14] [16] [15]

6. Prompt Engineering Best Practices

Sequential, chained prompts for coding

From Lovable, Cursor, and community "prompt lists," a few best practices emerge:

- Use explicit phases/modes:
  - Analysis mode: read codebase and ask clarifying questions, but do not change code.[9] [26]
  - Design/plan mode: propose a plan with numbered steps.[10] [11] [9]
  - Implementation mode: implement only the approved plan, in small, testable chunks.[26] [9] [10]
  - Test & docs mode: write or update tests, then update docs/README/CHANGELOG.[11] [26] [10]
- Scope and chunking:
  - Tools like Lovable explicitly recommend not trying to implement multiple unrelated features at once and to repeat key rules in each prompt.[9] [10] [11]
  - Using smaller, sequential prompts ("implement step 1," "now implement step 2") reduces regression risk.[26] [10]
- Context management:
  - Refer to specific files and paths; AI tools can search code faster than scanning giant rules files.[25] [10] [9]
  - Use knowledge/config files for stable rules and keep volatile context minimal, pruning regularly.[20] [23] [25] [10] [9]

Structuring system instructions & rules files

- .cursorrules best practices:
  - Cursor documentation and guides suggest keeping rules under ~1,000 tokens, clearly prioritizing key constraints and patterns.[20]
  - Organize by sections: Project overview, Architecture & conventions, Testing & quality, Prohibited behaviors, and Workflow (analyze, plan, implement).[23] [20] [9] [26]
- CLAUDE.md best practices:
  - Community advice: keep CLAUDE.md minimal and avoid putting code in it; move stable rules into dedicated `.claude/rules/` files and treat CLAUDE.md as a scratchpad for session context that's pruned.[25]
  - Explicit instructions telling Claude to scan rule files and load relevant ones into context can be added to CLAUDE.md.[23]
- Optimal structure for a full-stack React + Supabase project rules file:
  - Header: "Purpose & stack" (React + Vite/Next, Supabase, TypeScript, Tailwind, shadcn).
  - Architecture: folder structure, layering (components, hooks, services, db), and coding conventions (types first, no large files, maximum LOC per file).[20] [9] [26]
  - Data & Supabase: how to define tables, use Supabase client, and handle RLS/auth.[6] [19]
  - Testing: frameworks and required tests per change (unit tests for logic, smoke tests for screens).[10] [26]
  - Docs: where to update docs and what must be updated when adding features.[25] [26] [10]

Prompt chaining vs batching

- Prompt chaining: break a large goal into sequential prompts, each with clear sub-goals and review opportunities.[11] [26] [10]
- Prompt batching: try to include multiple steps in one long prompt.
- Observations from best-practice guides:
  - Lovable and community "super-prompt" posts push strongly towards chaining: smaller tasks, more validation, and explicit plan steps.[9] [26] [10] [11]
  - Batching leads to more context loss and regressions; users are advised not to implement multiple things at once.[10] [11]

Conclusion: Preflight should encode **prompt chaining by design**: automatically generate sequences of prompts with explicit stop-points and validation steps.

Open-source prompt libraries/frameworks

- While there is no canonical OSS "software-dev prompt framework," there are:
  - Collections of coding prompts shared in Reddit posts ("23 prompts I use for Cursor") which you can study and abstract.[26]
  - OSS projects providing AI config files and example rules (e.g., repos that share CLAUDE.md templates).[23] [25]

This is another opportunity for Preflight: ship an open "Prompt Packs" library for common stacks and workflows.

7. Open Source Launch Strategy

What drives 0 → 1,000 GitHub stars

From case studies and meta-posts:

- Solve a real, painful problem and show it clearly:
  - Successful tools often show a before/after demo and highlight painful workflows (e.g., "I hit 1k stars with an OSS code-doc generator by visualizing codebases").[24]
  - For dev-tools, stars correlate with community trust and visible value, but not directly with revenue.[21]
- Launch playbook:
  - High-quality README with clear positioning, GIFs, and "getting started in 2 minutes." [24] [6]
  - Demo video (YouTube, embedded in README) walking through a real project.[29] [24]
  - Coordinated launch: HN Show HN, Product Hunt, Reddit (r/Cursor, r/ClaudeCode, r/SaaS), and X thread.[29] [24] [6]
  - Early community involvement: ask for feedback on naming, logo, templates, and incorporate PRs quickly.[24] [6]
- Metrics:
  - Industry wisdom mentions 1,000 stars as a rough signal of solving a real problem for a real audience.[21]

Licensing choices for open-core

- MIT: very permissive, easy adoption, but no protections against closed-source competitors cloning your hosted version.
- Apache 2.0: like MIT but with explicit patent grants, popular for dev tools and may be more attractive for enterprise adoption.
- BSL/AGPL: more restrictive; BSL allows open-source usage but restricts commercial hosting; AGPL forces networked modifications to be open.[22]

For "OSS GitHub + paid hosted version":

- Many dev-tools use permissive licenses (MIT/Apache) for the core and keep premium features in a proprietary backend or "enterprise edition."[22] [6]
- If your priority is adoption among vibe coders and AI builders over strict protection, **MIT or Apache 2.0** is the most natural choice.[22] [6]

Monetization models for open-core dev tools

Common patterns:

- Free self-hosted + paid cloud hosting and enterprise features (SSO, team spaces, audit logs, collaboration).[22] [6]
- BYOK free + managed API paid: users can bring their own OpenAI/Claude keys for free; a managed mode with included credits and rate-limit protections is paid.[22]
- Support and consulting: custom integrations, prompt architecture services, and SLAs for enterprise customers.[22]

Recommendation:

- For Preflight:
  - OSS core (MIT/Apache): local-first app, BYOK, basic templates.[6] [22]
  - Hosted Preflight Cloud: multi-device sync, team spaces, collaboration on project OS, advanced templates, analytics, and managed AI credits.[6] [22]

Top recent OSS dev-tool launches to study

- Cursor (AI-native code editor) and related tools: massive growth and funding, built around AI collaboration.[6]
- Zed editor: Rust-based, open-source, and one of the most starred OSS projects from startups in 2024.[6]
- Ollama: open-source local model runner with huge star growth in 2024 (~261% to 136k+ stars).[6]
- Supabase: open-source Firebase alternative, widely adopted with strong docs and community.[6]
- Other dev-first products highlighted on Product Hunt in 2025 provide additional launch tactics.[29]

8. Key Risks & Blind Spots

- Rapid tool evolution:
  - AI IDEs like Cursor and Lovable are evolving quickly; they could introduce more powerful built-in project OS features that overlap with Preflight.[2] [10] [6]
- Over-complexity:
  - Vibe coders might resist heavy upfront configuration; Preflight must avoid feeling like "Jira for AI" and instead present itself as a fast, lightweight helper.
- Model & provider changes:

    - Changes in OpenAI/Anthropic/Google APIs or pricing may necessitate frequent updates to BYOK integrations.[5] [4]

- Adoption gap:
    - If Preflight requires too much discipline (e.g., manually updating stages), users might revert to "chat + Notion," limiting traction.

9. Recommended Next Steps

Short-term (0–4 weeks):

- Define 2–3 target archetypes (e.g., "solo founder shipping SaaS with Lovable + Supabase," "frontend dev using Cursor + Vercel").
- Ship a minimal Preflight MVP: React + Vite + Tailwind + shadcn/ui + Dexie + Zustand, with:
    - Project model (idea, goals, stack).
    - Generators for: research prompts, UI prompts, .cursorrules/CLAUDE.md, and a 5–10 step build sequence.[25] [20] [23] [11] [9] [26] [10]
    - Export actions to copy prompts/rules into AI editors.

Medium-term (1–3 months):

- Add a template library: Preflight Prompt Packs for common stacks (React + Supabase, Next + Prisma, etc.).[19] [16] [6]
- Implement BYOK integrations with a clean provider abstraction layer.
- Introduce optional Supabase sync (cloud mode) and shareable project links.[19] [6]

Launch (3–4 months):

- Prepare a polished README, demo video, and landing page clearly explaining "Preflight: Project OS for vibe coders." [24] [6]
- Coordinate launches on HN (Show HN), Product Hunt, Reddit (r/Cursor, r/ClaudeCode, r/SaaS), and X threads.[29] [24]
- Publish "Prompt architecture playbook for Cursor/Lovable" as content marketing, referencing Preflight templates.[11] [9] [26] [10]

Longer-term:

- Explore team features, analytics (e.g., prompt performance), and collaborative editing using ElectricSQL or CRDTs if necessary.[13] [19]
- Build a community around shared Preflight templates and prompt packs, encouraging PRs and sharing in AI-builder communities.[8] [24] [6]

⁂

1. https://www.youtube.com/watch?v=ltgksqJZT4M
2. https://techpoint.africa/guide/lovable-vs-bolt-vs-cursor/
3. https://replit.com/discover/best-vibe-coding-tools
4. https://www.abiresearch.com/news-resources/chart-data/report-artificial-intelligence-market-size-global

5. https://www.nextmsc.com/report/artificial-intelligence-market
6. https://www.landbase.com/blog/fastest-growing-open-source-dev-tools
7. https://www.precedenceresearch.com/artificial-intelligence-market
8. https://www.reddit.com/r/VibeCodeDevs/comments/1optl5r/whats_your_go_to_vibe_coding_platform_right_now/
9. https://www.reddit.com/r/LocalLLaMA/comments/1nlaefm/i_actually_read_four_system_prompts_from_cursor/
10. https://docs.lovable.dev/tips-tricks/best-practice
11. https://lovable.dev/blog/2025-01-16-lovable-prompting-handbook
12. https://dexie.org
13. https://rxdb.info/articles/zero-latency-local-first.html
14. https://shadcnstore.com/docs/dark-mode.html
15. https://blog.soards.me/posts/radix-colors-with-tailwind/
16. https://nishangiri.dev/blog/styling-react-apps
17. https://blog.logrocket.com/react-command-palette-tailwind-css-headless-ui/
18. https://www.reddit.com/r/reactjs/comments/1r1veob/built_a_keyboardfirst_job_tracker_ui_with_cmdk/
19. https://electric-sql.com/blog/2024/11/21/local-first-with-your-existing-api
20. https://www.agentrulegen.com/guides/cursorrules-vs-claude-md
21. https://www.getmonetizely.com/articles/what-role-should-github-stars-play-in-your-open-source-pricing-strategy-7cd11
22. https://www.reo.dev/blog/monetize-open-source-software
23. https://www.linkedin.com/pulse/configuring-claude-use-cursor-rules-robert-baillie-z2ybe
24. https://www.reddit.com/r/SaaS/comments/1p2od24/i_went_from_0_to_1k_github_stars_in_one_week/
25. https://www.reddit.com/r/ClaudeCode/comments/1riwy13/claudemd_best_practices/
26. https://www.reddit.com/r/cursor/comments/1la3bf0/23_prompts_i_use_for_flawless_cursor_code/
27. https://www.index.dev/blog/open-source-tools-for-developers
28. https://www.deployhq.com/blog/ai-coding-config-files-guide
29. https://dev.to/fmerian/5-awesome-dev-first-products-launched-on-product-hunt-in-2025-29i8
30. https://github.com/albingroen/react-cmdk